MARMARA ÜNİVERSİTESİ
Teknoloji Fakültesi, Bilgisayar Mühendisliği Bölümü

Mantık Devreleri

Ders 3: İkili Aritmetik

Deney yaprakları 1–4 için ek dosya

Ad Soyad	İsmail Efe Terlemez
Öğrenci No	171424002
Şube / Grup	A / Ali Bul
Tarih	29.10.2026

İçindekiler

Deney Yaprağı 1: İkili toplayıcı
Deney Yaprağı 2: Borç alma ve 2 tümleyeniyle çıkarma
Deney Yaprağı 3: Kısmi çarpımlarla çarpıcı
Deney Yaprağı 4: Bölüm, kalan ve kesirli bölme
Kısa cevaplar
Ek A: Kod

Deney Yaprağı 1

Elde adımlarını gösteren ikili toplayıcı

1. Bir sütunun sonuç ve elde tablosu

Her sütunda iki biti (a, b) ve sağdan gelen eldeyi (c) topluyorum. Toplam 0 ile 3 arasında olduğu için iki bitle yazılıyor: soldaki bit çıkan elde, sağdaki bit sonuç biti. Tabloyu kurduktan sonra şu ifadeleri çıkardım: $S = a \oplus b \oplus c$ ve $C_{out} = a \cdot b + c \cdot (a \oplus b)$.

a	b	c (gelen)	a + b + c	S (sonuç)	C_{out} (çıkan)
0	0	0	0	0	0
0	0	1	1	1	0
0	1	0	1	1	0
0	1	1	2	0	1
1	0	0	1	1	0
1	0	1	2	0	1
1	1	0	2	0	1
1	1	1	3	1	1

Algoritmayı sağdan sola çalışacak şekilde yazdım. Elde, başlangıç eldesiyle başlıyor; her i konumunda A_i, B_i ve gelen elde tam toplayıcıya giriyor, çıkan elde bir sonraki sütuna geçiyor. Her adımda ağırlığı (2^i), girdi bitlerini, gelen eldeyi, sonuç bitini ve çıkan eldeyi kaydediyorum. Döngü bittiğinde elimde kalan elde son eldedir. Kod Ek A'da: tam_toplayici() ve topla().

2. Dört işlem

Programı çalıştırmadan önce sonuçları elle hesapladım (“Elle beklenen” sütunu), sonra program çıktısıyla karşılaştırdım. İki sütunda da önce son elde, sonra 4 bitlik toplam yazıyor.

İşlem	Başl. elde	Elle beklenen	Program	Değer: C·16 + S	Uyuyor mu?
0101 + 0011	0	0 1000	0 1000	8 (5 + 3 + 0)	evet
1111 + 0001	0	1 0000	1 0000	16 (15 + 1 + 0)	evet
1111 + 0000	1	1 0000	1 0000	16 (15 + 0 + 1)	evet
1111 + 1111	1	1 1111	1 1111	31 (15 + 15 + 1)	evet

0101 + 0011, başlangıç eldesi 0: toplam 1000, son elde 0

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	0	1	0	1
B_i	0	0	1	1
gelen elde	1	1	1	0
sonuç S_i	1	0	0	0
çıkan elde	0	1	1	1

1111 + 0001, başlangıç eldesi 0: toplam 0000, son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	1	1	1	1
B_i	0	0	0	1
gelen elde	1	1	1	0
sonuç S_i	0	0	0	0
çıkan elde	1	1	1	1

1111 + 0000, başlangıç eldesi 1: toplam 0000, son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	1	1	1	1
B_i	0	0	0	0
gelen elde	1	1	1	1
sonuç S_i	0	0	0	0
çıkan elde	1	1	1	1

1111 + 1111, başlangıç eldesi 1: toplam 1111, son elde 1

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
A_i	1	1	1	1
B_i	1	1	1	1
gelen elde	1	1	1	1
sonuç S_i	1	1	1	1
çıkan elde	1	1	1	1

3. 512 test

0000 ile 1111 arasındaki bütün A, B çiftlerini iki başlangıç eldesiyle denedim: 16 × 16 × 2 = 512 test. Her testte programın verdiği değeri ($C \cdot 2^4 + S$), Python'un normal toplamasıyla bulduğum A + B + c0 ile karşılaştırdım. **512 testin hepsi tuttu, hata sayısı 0.**

4. Hatalı girdiler ve son eldenin anlamı

Şu durumlarda fonksiyon girdiyi reddedip ne olduğunu söyleyen bir hata mesajı veriyor: A veya B boşsa, içinde 0 ve 1 dışında bir karakter varsa, A ile B'nin uzunlukları farklıysa ve başlangıç eldesi 0 ya da 1 değilse. Her biri için bir test yaptım:

Durum	A	B	Başl. elde	Programın cevabı
boş girdi	(boş)	0101	0	ret: A: boş girdi kabul edilmez
geçersiz bit	01a1	0011	0	ret: A: geçersiz bit 'a' (yalnızca 0/1)
farklı uzunluk	101	0011	0	ret: farklı uzunluk: 3 ve 4 bit
geçersiz başlangıç eldesi	0101	0011	2	ret: başlangıç eldesi 0 ya da 1 olmalı

n bitlik bir toplamada gerçek sonuç A + B + c0 = $C \cdot 2^n + S$ olur. En büyük toplam $(2^n - 1) + (2^n - 1) + 1 = 2^{n+1} - 1$ olduğu için sonucu yazmak n + 1 bit gerektiriyor; bu fazladan bit son eldedir. Son eldeyi yok sayarsak elimizde yalnızca S kalıyor ve **$C \cdot 2^n$** kadar değer kayboluyor. C = 1 olduğunda kayıp tam olarak 2^n, yani n = 4 için 16.

İşlem	S	Son elde	Gerçek toplam	Elde atılınca	Kayıp
1111 + 0001 + 0	0000	1	16	0	$16 = 2^4$
1111 + 1111 + 1	1111	1	31	15	$16 = 2^4$
1000 + 1000 + 0	0000	1	16	0	$16 = 2^4$

Sınır örneği olarak 1111 + 0001'i seçtim: doğru sonuç 1 0000 (16), ama son elde atılınca 0000 kalıyor. Yani iki pozitif sayının toplamı sıfır gibi görünüyor. İşaretsiz toplamada taşma olduğunu bize son elde gösteriyor.

Deney Yaprağı 2

Borç alma ve 2 tümleyeniyle çıkarma

1. İki ayrı algoritma

Borç alarak çıkarma (cikar_borc): Sağdan sola her sütunda fark biti D = m ⊕ n ⊕ b, çıkan borç B_{out} = m'·n + (m ⊕ n)'·b. İşlem bitince borç kalmadıysa M ≥ N'dir ve fark doğrudan sonuçtur. Fark 0000 çıkarsa sonucu işaretsiz “0” olarak veriyorum. Son borç 1 ise M < N'dir; sonuca eksi işareti koyuyorum ve büyüklüğü aynı yöntemle N − M hesaplayarak buluyorum.

2 tümleyeniyle çıkarma (cikar_tumleyen): M − N = M + N' + 1. N'nin bitlerini ters çeviriyorum ve +1'i toplayıcıya başlangıç eldesi olarak veriyorum. Bu fonksiyonun kendi toplayıcı döngüsü var, cikar_borc'u hiç çağırmıyor. Son elde 1 ise M ≥ N ve sonuç toplamın kendisi. Son elde 0 ise M < N; büyüklüğü bulmak için toplamın 2 tümleyenini alıyorum (bitleri ters çevirip 1 ekliyorum).

2. Beş işlem

İşlem	Onlu	Borç yöntemi: son borç	Borç yöntemi: sonuç	2 tümleyeni: son elde	2 tümleyeni: sonuç	Doğru sonuç
1110 − 0011	14 − 3	0	+1011	1	+1011	11
0011 − 1110	3 − 14	1	−1011	0	−1011	−11
1000 − 0001	8 − 1	0	+0111	1	+0111	7
0101 − 0101	5 − 5	0	0	1	0	0
0011 − 0000	3 − 0	0	+0011	1	+0011	3

Her işlemin ara kayıtları aşağıda. Tablolarda sütunlar bit konumlarını, satırlar o konumdaki bitleri, borcu veya eldeyi gösteriyor.

1110 − 0011

Borç zinciri (M − N):

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	1	1	0
N_i	0	0	1	1
gelen borç	0	1	1	0
fark D_i	1	0	1	1
çıkan borç	0	0	1	1

2 tümleyeni: N' = 1100, M + N' + 1 (başlangıç eldesi 1), son elde 1:

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	1	1	0
N'_i	1	1	0	0
gelen elde	1	0	0	1
toplam	1	0	1	1
çıkan elde	1	1	0	0

Sonuç: **+1011** (onlu 11). İki yöntem aynı sonucu verdi.

0011 − 1110

Borç zinciri (M − N):

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N_i	1	1	1	0
gelen borç	1	0	0	0
fark D_i	0	1	0	1
çıkan borç	1	1	0	0

Son borç 1 çıktı, sonuç negatif. Büyüklük için N − M:

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
N_i	1	1	1	0
M_i	0	0	1	1
gelen borç	0	1	1	0
fark D_i	1	0	1	1
çıkan borç	0	0	1	1

2 tümleyeni: N' = 0001, M + N' + 1 (başlangıç eldesi 1), son elde 0:

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N'_i	0	0	0	1
gelen elde	0	1	1	1
toplam	0	1	0	1
çıkan elde	0	0	1	1

Son elde 0 çıktı, sonuç negatif. Büyüklük = ters çevrilmiş toplam + 1:

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
ters çevr. toplam	1	0	1	0
eklenen	0	0	0	0
gelen elde	0	0	0	1
büyüklük	1	0	1	1
çıkan elde	0	0	0	0

Sonuç: **−1011** (onlu −11). İki yöntem aynı sonucu verdi.

1000 − 0001

Borç zinciri (M − N):

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	0	0	0
N_i	0	0	0	1
gelen borç	1	1	1	0
fark D_i	0	1	1	1
çıkan borç	0	1	1	1

2 tümleyeni: N' = 1110, M + N' + 1 (başlangıç eldesi 1), son elde 1:

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	1	0	0	0
N'_i	1	1	1	0
gelen elde	0	0	0	1
toplam	0	1	1	1
çıkan elde	1	0	0	0

Sonuç: **+0111** (onlu 7). İki yöntem aynı sonucu verdi.

0101 − 0101

Borç zinciri (M − N):

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	1	0	1
N_i	0	1	0	1
gelen borç	0	0	0	0
fark D_i	0	0	0	0
çıkan borç	0	0	0	0

2 tümleyeni: N' = 1010, M + N' + 1 (başlangıç eldesi 1), son elde 1:

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	1	0	1
N'_i	1	0	1	0
gelen elde	1	1	1	1
toplam	0	0	0	0
çıkan elde	1	1	1	1

Sonuç: **0** (onlu 0). İki yöntem aynı sonucu verdi.

0011 − 0000

Borç zinciri (M − N):

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N_i	0	0	0	0
gelen borç	0	0	0	0
fark D_i	0	0	1	1
çıkan borç	0	0	0	0

2 tümleyeni: N' = 1111, M + N' + 1 (başlangıç eldesi 1), son elde 1:

Konum i	3	2	1	0
Ağırlık 2^i	8	4	2	1
M_i	0	0	1	1
N'_i	1	1	1	1
gelen elde	1	1	1	1
toplam	0	0	1	1
çıkan elde	1	1	1	1

Sonuç: **+0011** (onlu 3). İki yöntem aynı sonucu verdi.

3. 256 çiftin hepsi

M ve N için 0–15 aralığındaki bütün 256 çifti iki yöntemle de hesapladım ve sonucu Python'da M − N ile karşılaştırdım. Sonucun 0 olduğu durumlarda işaretsiz “0” verilip verilmediğine de baktım. Borç yönteminde 0, 2 tümleyeni yönteminde 0 hata çıktı. Uyumsuz çift olmadığı için düzeltme gerekmedi.

4. “Son elde yoksa sonuç mutlaka negatiftir” iddiası

N'nin 2 tümleyenini ayrı bir adımda ve 4 bitle hesaplarsak: 0000'ı ters çevirince 1111, 1 ekleyince 1 0000 oluyor. 4 bite sığmayan 1 atılıyor ve 0000'ın 2 tümleyeni yine **0000** çıkıyor. Bundan sonra M + 0000 toplamında hiç elde oluşmuyor.

Bu şekilde (önce tümleyeni ayrı al, sonra eldesiz topla) çalışan bir sürüm yazıp 256 çifti denedim. İddia 16 çiftte yanlış karar verdi ve hepsinde N = 0000 idi. **En küçük karşı örnek 0000 − 0000:** toplam 0000, son elde 0, ama sonuç 0 ve negatif değil. Aynı şekilde 0011 − 0000'da da elde yok, sonuç ise +3.

Doğru karar kuralı: çıkarmayı tek seferde M + N' + 1 olarak yapmak, yani +1'i başlangıç eldesi olarak vermek. N = 0 iken N' = 1111 olur ve 1111 + 1 her zaman elde üretir. Programda M = 0000, 0011 ve 1111 için N = 0 denedim, üçünde de son elde 1 çıktı. Kural şu: **son elde 1 ise M ≥ N** (sonuç pozitif veya sıfır, büyüklük toplamın kendisi), **son elde 0 ise M < N** (sonuç negatif, büyüklük toplamın 2 tümleyeni). Tümleyen ayrı hesaplanacaksa N = 0 durumunu ayrıca ele almak gerekiyor.

Deney Yaprağı 3

Kısmi çarpımlarla ikili çarpıcı

1. Kısmi çarpım ve kaydırma kuralı

Çarpanın i konumundaki biti b_i ise (bitleri sağdan 0'dan başlayarak numaralandırdım) kısmi çarpım $PP_i = b_i \cdot A \cdot 2^i$ olur. $b_i = 1$ ise A'yı, 0 ise n tane sıfırı alıyorum, sağına i tane 0 ekleyerek i kez sola kaydırıyorum ve soldan sıfırla doldurup n + m bite tamamlıyorum. Sonra kısmi çarpımları Yaprak 1'deki toplayıcıyla n + m bitlik bir toplama ekliyorum. Asıl hesapta Python'un çarpmasını veya sayıya çevirmeyi kullanmadım. Kod Ek A'da: kismi_carpim() ve carp().

2. Kısmi çarpım tabloları

1011 × 101 (n = 4, m = 3, sonuç 7 bit): **0110111**. Kontrol için onlu çarptım: 11 × 5 = 55, tuttu.

i	b_i	PP_i (7 bit)	Onlu katkı
0	1	0001011	$1 \cdot 11 \cdot 2^0 = 11$
1	0	0000000	$0 \cdot 11 \cdot 2^1 = 0$
2	1	0101100	$1 \cdot 11 \cdot 2^2 = 44$
	Toplam	**0110111**	**11 + 0 + 44 = 55**

10111 × 110 (n = 5, m = 3, sonuç 8 bit): **10001010**. Kontrol için onlu çarptım: 23 × 6 = 138, tuttu.

i	b_i	PP_i (8 bit)	Onlu katkı
0	0	00000000	$0 \cdot 23 \cdot 2^0 = 0$
1	1	00101110	$1 \cdot 23 \cdot 2^1 = 46$
2	1	01011100	$1 \cdot 23 \cdot 2^2 = 92$
	Toplam	**10001010**	**0 + 46 + 92 = 138**

11001 × 1010 (n = 5, m = 4, sonuç 9 bit): **011111010**. Kontrol için onlu çarptım: 25 × 10 = 250, tuttu.

i	b_i	PP_i (9 bit)	Onlu katkı
0	0	000000000	$0 \cdot 25 \cdot 2^0 = 0$
1	1	000110010	$1 \cdot 25 \cdot 2^1 = 50$
2	0	000000000	$0 \cdot 25 \cdot 2^2 = 0$
3	1	011001000	$1 \cdot 25 \cdot 2^3 = 200$
	Toplam	**011111010**	**0 + 50 + 0 + 200 = 250**

3. Testler ve sıfır satırları atlayan ikinci sürüm

Toplama sayısını şöyle saydım: toplam 0 ile başlıyor ve ona eklenen her kısmi çarpım bir toplama sayılıyor. Birinci sürüm çarpanın bütün bitlerini işlediği için m toplama yapıyor. İkinci sürüm $b_i = 0$ olan satırları atlıyor, bu yüzden çarpandaki 1 sayısı kadar toplama yapıyor (çarpan 0 ise hiç).

Test	A	B	Sürüm 1	Sürüm 2	Toplama sayısı 1	Toplama sayısı 2	Onlu	Uyuyor mu?
çarpan sıfır	1011	0	00000	00000	1	0	0	evet
çarpan bir	1011	1	01011	01011	1	1	11	evet
çarpan 2^2	1011	100	0101100	0101100	3	1	44	evet
çarpan 2^3	101	1000	0101000	0101000	4	1	40	evet
çarpan tüm 1	1011	111	1001101	1001101	3	3	77	evet
iki operand tüm 1	1111	1111	11100001	11100001	4	4	225	evet
çarpılan sıfır	0	101	0000	0000	3	2	0	evet
1 × 1	1	1	01	01	1	1	1	evet
n+m-1 karşı örneği	11	11	1001	1001	2	2	9	evet
seyrek çarpan	110	10001	01100110	01100110	5	2	102	evet

İki sürüm bütün testlerde aynı sonucu verdi, sadece toplama sayıları farklı. Sıfır satırı eklemek sonucu değiştirmediği için atlamak güvenli.

4. Sonuç neden n + m bit?

n bitlik en büyük sayı $2^n - 1$, m bitlik en büyük sayı $2^m - 1$. Bunları çarpınca $(2^n - 1)(2^m - 1) = 2^{n+m} - 2^n - 2^m + 1$ çıkıyor ve bu 2^{n+m}'den küçük. Yani en büyük çarpım bile n + m bite sığıyor. Örneğin 1111 × 1111 = 225 = 11100001 (8 bit).

n + m − 1 bit ise her zaman yetmiyor: **11 × 11 = 3 × 3 = 9 = 1001**. Burada n + m − 1 = 3 bit ve 3 bitle en fazla 7 yazılabiliyor, 9 için 4 bit lazım. n veya m 1 olduğunda çarpım $2^n - 1$'i geçmediği için n + m − 1 bit yetiyor; bu yüzden en küçük karşı örnek n = m = 2 ile çıkıyor.

Deney Yaprağı 4

Bölüm, kalan ve kesirli bölme algoritması

1. Soldan sağa bölme

Kalanı 0 ile başlatıyorum ve bölünenin bitlerini soldan sağa tek tek indiriyorum. İncelenen parça, kalanın sağına sıradaki bitin eklenmesiyle oluşuyor. Parça bölenden büyük veya eşitse bölüme 1 yazıyorum ve parçadan böleni çıkarıyorum (Yaprak 2'deki borçlu çıkarma). Değilse bölüme 0 yazıyorum ve parça olduğu gibi kalan oluyor. Karşılaştırmayı da bit düzeyinde yaptım: baştaki sıfırları atıp önce uzunluklara, uzunluklar eşitse soldan ilk farklı bite bakıyorum. Bölen sıfırsa fonksiyon hata verip duruyor. Asıl hesapta hazır bölme ve kalan operatörü yok. Kod Ek A'da: buyuk_esit() ve bol().

2 ve 3. Üç işlem, bölüm/kalan kontrolü ve kesirli devam

10110 ÷ 100 (22 ÷ 4)

Adım	İncelenen parça	Bölüm biti	Kalan
1	1	0	1
2	10	0	10
3	101	1	1
4	11	0	11
5	110	1	10

Q = **101** (5), R = **10** (2). Birinci kontrol: Q·d + R = 5·4 + 2 = 22, bu D'ye eşit. İkinci kontrol: 0 ≤ R = 2 < d = 4, bu da sağlanıyor. Python'un // ve % operatörleri de Q = 5, R = 2 verdi.

Kesir adımı	İncelenen parça	Bölüm biti	Kalan
1	100	1	0

Kesirli sonuç: 10110 ÷ 100 = **101.1** (ikili). Onlu olarak 5.5; gerçek değer 5.5. Kalan sıfırlandığı için kesirli kısım burada bitti, fazladan sıfır eklemedim.

1101 ÷ 10 (13 ÷ 2)

Adım	İncelenen parça	Bölüm biti	Kalan
1	1	0	1
2	11	1	1
3	10	1	0
4	1	0	1

Q = **110** (6), R = **1** (1). Birinci kontrol: Q·d + R = 6·2 + 1 = 13, bu D'ye eşit. İkinci kontrol: 0 ≤ R = 1 < d = 2, bu da sağlanıyor. Python'un // ve % operatörleri de Q = 6, R = 1 verdi.

Kesir adımı	İncelenen parça	Bölüm biti	Kalan
1	10	1	0

Kesirli sonuç: 1101 ÷ 10 = **110.1** (ikili). Onlu olarak 6.5; gerçek değer 6.5. Kalan sıfırlandığı için kesirli kısım burada bitti, fazladan sıfır eklemedim.

111 ÷ 11 (7 ÷ 3)

Adım	İncelenen parça	Bölüm biti	Kalan
1	1	0	1
2	11	1	0
3	1	0	1

Q = **10** (2), R = **1** (1). Birinci kontrol: Q·d + R = 2·3 + 1 = 7, bu D'ye eşit. İkinci kontrol: 0 ≤ R = 1 < d = 3, bu da sağlanıyor. Python'un // ve % operatörleri de Q = 2, R = 1 verdi.

Kesir adımı	İncelenen parça	Bölüm biti	Kalan
1	10	0	10
2	100	1	1
3	10	0	10
4	100	1	1
5	10	0	10
6	100	1	1
7	10	0	10
8	100	1	1

Kesirli sonuç: 111 ÷ 11 ≈ **10.01010101** (ikili). Onlu olarak 2.3320312; gerçek değer 2.3333333. Kalanlar 1, 10, 1, 10 diye tekrar ediyor; bir kalan ikinci kez gelince basamaklar da tekrar etmeye başlıyor (“01” devri). Kesir hiç bitmiyor, 8 basamakta kestim.

İki kontrol neden birlikte gerekli?

Sadece D = Q·d + R'ye bakmak yetmiyor. 10110 ÷ 100 için Q = 100 (4), R = 110 (6) alırsam 4·4 + 6 = 22 tutuyor ama R ≥ d olduğu için bölme bitmemiş demektir (Q = 0, R = D zaten her zaman tutar). Sadece 0 ≤ R < d'ye bakmak da yetmiyor: Q = 100, R = 10 bu koşulu sağlıyor ama 4·4 + 2 = 18, 22 değil. İki koşul birlikte sağlanınca Q ve R tek türlü belirleniyor.

Durma koşulları (111 ÷ 11)

Tam bölme, bölünende indirilecek bit kalmayınca duruyor. 111'in 3 biti olduğu için 3 adımda Q = 10, R = 1 kesinleşiyor. Kesirli bölmede ise kalanın sağına 0 ekleyerek devam ediyorum. Kalan 0 olursa sonuç tamamlanmış oluyor ve “=” yazıyorum; 8 basamağa ulaşırsam kesip “≈” yazıyorum. 111 ÷ 11'de kalanlar 1 ve 10 arasında gidip geliyor, hiç 0 olmuyor: 7/3 = 10.0101... (ikili), 8 basamakla 10.01010101. 10110 ÷ 100 ve 1101 ÷ 10'da ise ilk kesir basamağında kalan 0 olduğu için sonuç “=” ile bitti.

4. Sınır testleri

Durum	D	d	Beklenen Q, R	Program	Uyuyor mu?
sıfır bölünen	0	101	0, 0	0, 0	evet
sıfır bölünen (3 bit)	000	11	0, 0	0, 0	evet
bölen > bölünen	11	101	0, 11	0, 11	evet
eşit operandlar	101	101	1, 0	1, 0	evet
eşit operandlar	1011	1011	1, 0	1, 0	evet
sıfır bölen	1010	000	ret	ret: bölen sıfır: bölme tanımsız, girdi reddedildi	evet

Bölümde sıfır bitinin yanlışlıkla atılmasını göstermek için bol_hatali() diye bir sürüm yazdım: bölen sığmadığında bölüme 0 yazmayı unutuyor. Baştaki sıfırlarda bunun zararı yok, ama aradaki sıfırlar kaybolunca bölüm değişiyor:

D ÷ d	Doğru Q	Hatalı Q	Hatalı R	Q·d + R kontrolü
10110 ÷ 100	101	11	10	3·4 + 2 = 14, D = 22: hata yakalandı
1101 ÷ 10	110	11	1	3·2 + 1 = 7, D = 13: hata yakalandı
100100 ÷ 11	1100	11	0	3·3 + 0 = 9, D = 36: hata yakalandı

Örneğin 10110 ÷ 100'de doğru bölüm 101. Ortadaki 0 atılınca 11 (3) çıkıyor ve 3·4 + 2 = 14 olduğu için D = Q·d + R kontrolü hatayı yakalıyor. Kalan doğru kaldığı için R < d kontrolü tek başına bunu fark edemezdi; bu da iki kontrolün neden birlikte gerektiğine bir örnek.

Kısa cevaplar

Deney yapraklarındaki “Kısa cevap ve doğrulama notları” alanlarına yazdıklarım

Deney Yaprağı 1

İlk tahminim	0101 + 0011 = 1000, elde 0. 1111 + 0001 ve 1111 + 0000 + 1 için 0000 ve elde 1, 1111 + 1111 + 1 için 1111 ve elde 1 bekledim.
Bağımsız kontrolüm	512 testin hepsinde elde·16 + toplamı Python'un normal toplamasıyla karşılaştırdım, hiç hata çıkmadı.
Karşı örneğim	1111 + 0001'de son elde atılırsa sonuç 0000 kalıyor, yani 16 kayboluyor. Genel olarak kayıp elde·2^n.
YZ beyanı	Claude'dan kod taslağı aldım. Dört işlemi önce elle çözdüm, sonra 512 testle karşılaştırdım.

Deney Yaprağı 2

İlk tahminim	1110 − 0011 = +1011, 0011 − 1110 = −1011, 1000 − 0001 = +0111, 0101 − 0101 = 0, 0011 − 0000 = +0011 olur diye düşündüm.
Bağımsız kontrolüm	256 çiftin hepsini iki yöntemle ayrı ayrı hesaplayıp Python'daki M − N ile karşılaştırdım. İkisinde de hata çıkmadı.
Karşı örneğim	0000 − 0000: 0000'ın 2 tümleyeni yine 0000, toplamada elde çıkmıyor ama sonuç negatif değil, 0. Bu yüzden +1'i başlangıç eldesi olarak verdim.
YZ beyanı	YZ ile kod yazdım. N = 0 durumunu ayrıca denedim, karar kuralını buna göre kurdum.

Deney Yaprağı 3

İlk tahminim	1011 × 101 = 55 = 0110111, 10111 × 110 = 138 = 10001010, 11001 × 1010 = 250 = 011111010 çıkacağını hesapladım.
Bağımsız kontrolüm	Kısmi çarpımların onlu değerlerini elle topladım (11 + 0 + 44 = 55 gibi) ve 10 testte iki sürümü onlu çarpımla karşılaştırdım.
Karşı örneğim	11 × 11 = 1001 (9). n + m − 1 = 3 bitle en fazla 7 yazılabildiği için 3 bit yetmiyor.
YZ beyanı	Claude'dan kod taslağı aldım. Üç kısmi çarpım tablosunu elle toplayarak kontrol ettim.

Deney Yaprağı 4

İlk tahminim	10110 ÷ 100 için Q = 101, R = 10; 1101 ÷ 10 için Q = 110, R = 1; 111 ÷ 11 için Q = 10, R = 1 ve kesrin bitmeyeceğini düşündüm.
Bağımsız kontrolüm	Her işlemde Q·d + R = D ve R < d koşullarına ayrı ayrı baktım, sonuçları Python'un // ve % operatörleriyle karşılaştırdım.
Karşı örneğim	Aradaki sıfırı atan sürüm 10110 ÷ 100 için Q = 11 veriyor: 3·4 + 2 = 14, 22 değil. Q·d + R kontrolü bunu yakalıyor.
YZ beyanı	YZ ile kod yazdım. Sıfır böleni ve sınır durumlarını kendim deneyip sonuçları kontrol ettim.

Ek A: Kod (Python 3)

ikili.py dört yaprağın algoritmalarını, testler.py istenen testleri içeriyor. Testleri çalıştırmak için: python3 testler.py

ikili.py

```
"""Mantık Devreleri - Ders 3 / İkili Aritmetik deney yaprakları 1-4.

Ana hesaplar yalnızca bit düzeyinde (karakter '0'/'1' ve mantık işlemleri) yapılır.
Hazır aritmetik (int(), +, -, *, //, %) yalnızca test referanslarında ve
onlu gösterimlerde kullanılır.
"""

# ---------------------------------------------------------------------------
# Ortak yardımcılar
# ---------------------------------------------------------------------------

def dogrula(bitler, ad="girdi"):
    """Boş girdi ve geçersiz bit için ret kuralı."""
    if not isinstance(bitler, str) or len(bitler) == 0:
        raise ValueError(f"{ad}: boş girdi kabul edilmez")
    for ch in bitler:
        if ch not in "01":
            raise ValueError(f"{ad}: geçersiz bit '{ch}' (yalnızca 0/1)")


def tersle(bitler):
    """Her biti tersler (1 tümleyeni)."""
    return "".join("1" if b == "0" else "0" for b in bitler)


def sola_doldur(bitler, genislik):
    return "0" * (genislik - len(bitler)) + bitler


def bas_sifir_at(bitler):
    i = 0
    while i < len(bitler) - 1 and bitler[i] == "0":
        i += 1
    return bitler[i:]


# ---------------------------------------------------------------------------
# Yaprak 1 - Elde adımlarını gösteren ikili toplayıcı
# ---------------------------------------------------------------------------

def tam_toplayici(a, b, c):
    """Bir sütun: (sonuç biti, çıkan elde). a, b, c in {0, 1}."""
    s = a ^ b ^ c
    c_out = (a & b) | (c & (a ^ b))
    return s, c_out


def dogruluk_tablosu():
    satirlar = []
    for a in (0, 1):
        for b in (0, 1):
            for c in (0, 1):
                s, co = tam_toplayici(a, b, c)
                satirlar.append((a, b, c, s, co))
    return satirlar


def topla(A, B, elde0=0):
    """n bitlik A + B + elde0. Döner: (toplam, son_elde, sütun_kaydı).

    sütun_kaydı satırı: (ağırlık i, a_i, b_i, gelen elde, sonuç biti, çıkan elde)
    """
    dogrula(A, "A")
    dogrula(B, "B")
    if len(A) != len(B):
        raise ValueError(f"farklı uzunluk: {len(A)} ve {len(B)} bit")
    if elde0 not in (0, 1):
        raise ValueError("başlangıç eldesi 0 ya da 1 olmalı")
    n = len(A)
    c = elde0
    sonuc = [""] * n
    kayit = []
    for i in range(n):              # sağdan sola: ağırlık 2^i
        a = 1 if A[n - 1 - i] == "1" else 0
        b = 1 if B[n - 1 - i] == "1" else 0
```

```
        s, co = tam_toplayici(a, b, c)
        kayit.append((i, a, b, c, s, co))
        sonuc[n - 1 - i] = "1" if s else "0"
        c = co
    return "".join(sonuc), c, kayit


# ---------------------------------------------------------------------------
# Yaprak 2 - Borç alma ve 2 tümleyeniyle çıkarma
# ---------------------------------------------------------------------------

def _borcla_cikar_ham(M, N):
    """Sabit genişlikte M - N, borç alarak. Döner: (fark, son_borç, kayıt)."""
    n = len(M)
    borc = 0
    fark = [""] * n
    kayit = []
    for i in range(n):
        m = 1 if M[n - 1 - i] == "1" else 0
        k = 1 if N[n - 1 - i] == "1" else 0
        d = m ^ k ^ borc
        b_out = ((1 - m) & k) | ((1 - (m ^ k)) & borc)
        kayit.append((i, m, k, borc, d, b_out))
        fark[n - 1 - i] = "1" if d else "0"
        borc = b_out
    return "".join(fark), borc, kayit


def cikar_borc(M, N):
    """Doğrudan çıkarma. Döner: (işaret, büyüklük, kayıtlar).

    Son borç 1 ise M < N'dir; büyüklük aynı yöntemle N - M hesaplanarak bulunur.
    """
    dogrula(M, "M")
    dogrula(N, "N")
    if len(M) != len(N):
        raise ValueError("farklı uzunluk")
    fark, borc, kayit = _borcla_cikar_ham(M, N)
    kayitlar = {"M-N": kayit, "son_borc": borc}
    if borc == 0:
        # sıfır durumu ayrıca: işaretsiz "0"
        return ("+" if "1" in fark else "0"), fark, kayitlar
    buyukluk, _, kayit2 = _borcla_cikar_ham(N, M)
    kayitlar["N-M"] = kayit2
    return "-", buyukluk, kayitlar


def _tumleyen_topla_ham(X, Y, elde0):
    """2 tümleyeni yöntemi için kendi tam toplayıcı zinciri (cikar_borc'u çağırmaz)."""
    n = len(X)
    c = elde0
    s = [""] * n
    kayit = []
    for i in range(n):
        a = 1 if X[n - 1 - i] == "1" else 0
        b = 1 if Y[n - 1 - i] == "1" else 0
        t, co = tam_toplayici(a, b, c)
        kayit.append((i, a, b, c, t, co))
        s[n - 1 - i] = "1" if t else "0"
        c = co
    return "".join(s), c, kayit


def cikar_tumleyen(M, N):
    """M - N = M + (N'nin 1 tümleyeni) + 1. '+1' başlangıç eldesi olarak girer.

    Son elde 1 -> M >= N, sonuç doğrudan toplamdır.
    Son elde 0 -> M < N, büyüklük = toplamın 2 tümleyeni.
    """
    dogrula(M, "M")
    dogrula(N, "N")
    if len(M) != len(N):
        raise ValueError("farklı uzunluk")
    n = len(M)
    N1 = tersle(N)
    toplam, son_elde, kayit = _tumleyen_topla_ham(M, N1, 1)
    kayitlar = {"N_1tum": N1, "M+N'+1": kayit, "son_elde": son_elde}
    if son_elde == 1:
        if "1" in toplam:
            return "+", toplam, kayitlar
        return "0", toplam, kayitlar
    T1 = tersle(toplam)
    buyukluk, _, kayit2 = _tumleyen_topla_ham(T1, "0" * n, 1)
    kayitlar["buyukluk"] = kayit2
```

```
    return "-", buyukluk, kayitlar


def iki_tumleyeni(N):
    """Sabit genişlikte 2 tümleyeni (taşan elde atılır). Döner: (sonuç, atılan elde)."""
    s, c, _ = _tumleyen_topla_ham(tersle(N), "0" * len(N), 1)
    return s, c


def cikar_tumleyen_hatali(M, N):
    """Hatalı yaklaşım: önce 2 tümleyenini ayrı hesaplayıp kesmek, sonra eldesiz toplamak.
    'Son elde yoksa negatif' kuralı burada N = 0 için yanlış karar verir."""
    N2, _ = iki_tumleyeni(N)
    toplam, son_elde, _ = _tumleyen_topla_ham(M, N2, 0)
    return ("+" if son_elde == 1 else "-"), toplam, son_elde


# ---------------------------------------------------------------------------
# Yaprak 3 - Kısmi çarpımlarla ikili çarpıcı
# ---------------------------------------------------------------------------

def kismi_carpim(A, b_i, i, genislik):
    """i konumundaki çarpan biti için: b_i = 1 ise A, değilse 0...0;
    sağa i adet 0 eklenerek i kez sola kaydırılır, n+m bite sola sıfırla doldurulur."""
    taban = A if b_i == "1" else "0" * len(A)
    return sola_doldur(taban + "0" * i, genislik)


def carp(A, B, sifirlari_atla=False):
    """A (n bit) x B (m bit) -> n+m bit. Döner: (çarpım, kısmi_çarpımlar, toplama_sayısı).

    Sayım kuralı: biriktirici 0...0 ile başlar; biriktiriciye eklenen her kısmi
    çarpım bir toplama çağrısıdır. v1: m çağrı, v2 (sıfırları atla): B'deki 1 sayısı.
    """
    dogrula(A, "çarpılan")
    dogrula(B, "çarpan")
    n, m = len(A), len(B)
    g = n + m
    acc = "0" * g
    kismiler = []
    cagri = 0
    for i in range(m):
        b_i = B[m - 1 - i]
        pp = kismi_carpim(A, b_i, i, g)
        kismiler.append((i, b_i, pp))
        if sifirlari_atla and b_i == "0":
            continue
        acc, son_elde, _ = topla(acc, pp, 0)
        cagri += 1
        assert son_elde == 0, "n+m bit yetmedi (olmamalı)"
    return acc, kismiler, cagri


# ---------------------------------------------------------------------------
# Yaprak 4 - Bölüm, kalan ve kesirli bölme
# ---------------------------------------------------------------------------

def buyuk_esit(X, Y):
    """X >= Y ? (baştaki sıfırlar atılarak; önce uzunluk, sonra soldan ilk farklı bit)."""
    X, Y = bas_sifir_at(X), bas_sifir_at(Y)
    if len(X) != len(Y):
        return len(X) > len(Y)
    for x, y in zip(X, Y):
        if x != y:
            return x == "1"
    return True


def _fark(X, Y):
    """X >= Y varsayımıyla X - Y (borçlu çıkarma), baştaki sıfırlar atılmış."""
    g = max(len(X), len(Y))
    f, borc, _ = _borcla_cikar_ham(sola_doldur(X, g), sola_doldur(Y, g))
    assert borc == 0
    return bas_sifir_at(f)


def bol(D, d, kesir_basamak=0):
    """Soldan sağa uzun bölme. Döner: sözlük (Q, R, adimlar, kesir, kesir_adim, tam_mi)."""
    dogrula(D, "bölünen")
    dogrula(d, "bölen")
    if "1" not in d:
        raise ZeroDivisionError("bölen sıfır: bölme tanımsız, girdi reddedildi")
    if not (0 <= kesir_basamak <= 8):
        raise ValueError("kesir basamağı 0..8 olmalı")
```

```
    r = "0"
    q = []
    adimlar = []
    for j, bit in enumerate(D):
        parca = bas_sifir_at(r + bit)          # kalanı sola kaydır, sıradaki biti indir
        if buyuk_esit(parca, d):
            qb = "1"
            r = _fark(parca, d)
        else:
            qb = "0"
            r = parca
        q.append(qb)
        adimlar.append((j + 1, parca, qb, r))
    Q = bas_sifir_at("".join(q))
    R = r
    kesir = []
    kesir_adim = []
    gorulen = {R: 0}
    tekrar = None
    while R != "0" and len(kesir) < kesir_basamak:
        parca = bas_sifir_at(R + "0")
        if buyuk_esit(parca, d):
            fb = "1"
            R = _fark(parca, d)
        else:
            fb = "0"
            R = parca
        kesir.append(fb)
        kesir_adim.append((len(kesir), parca, fb, R))
        if R in gorulen and tekrar is None and R != "0":
            tekrar = (gorulen[R], len(kesir), R)
        gorulen.setdefault(R, len(kesir))
    return {
        "Q": Q, "R": r, "adimlar": adimlar,
        "kesir": "".join(kesir), "kesir_adim": kesir_adim,
        "tam_mi": R == "0", "tekrar": tekrar,
    }


def bol_hatali(D, d):
    """Hatalı sürüm: bölen sığmadığında bölüme 0 yazmayı atlar (sıfır biti düşer)."""
    r = "0"
    q = []
    for bit in D:
        parca = bas_sifir_at(r + bit)
        if buyuk_esit(parca, d):
            q.append("1")
            r = _fark(parca, d)
        else:
            r = parca                       # HATA: q.append("0") unutuldu
    return bas_sifir_at("".join(q) or "0"), r
```

testler.py

```
"""Deney 3 testleri. Referans değerler hazır aritmetikle (int, +, -, *, //, %) hesaplanır.

Çalıştırma:  python3 testler.py
"""
from itertools import product

from ikili import (topla, dogruluk_tablosu, cikar_borc, cikar_tumleyen,
                   cikar_tumleyen_hatali, iki_tumleyeni, carp, bol, bol_hatali)


def ikili(x, n):
    return format(x, f"0{n}b")


def onlu_isaretli(isaret, buyukluk):
    v = int(buyukluk, 2)
    return -v if isaret == "-" else v


# ------------------------------ Yaprak 1 ---------------------------------

def yaprak1():
    sonuc = {"tablo": dogruluk_tablosu()}
    islemler = [("0101", "0011", 0, "1000", 0), ("1111", "0001", 0, "0000", 1),
                ("1111", "0000", 1, "0000", 1), ("1111", "1111", 1, "1111", 1)]
    kayitlar = []
    for A, B, c0, beklenen_s, beklenen_c in islemler:
        s, c, kayit = topla(A, B, c0)
        kayitlar.append({"A": A, "B": B, "c0": c0, "S": s, "C": c, "kayit": kayit,
                         "beklenen": (beklenen_s, beklenen_c),
```

```
                        "uyum": (s, c) == (beklenen_s, beklenen_c)})
    sonuc["islemler"] = kayitlar

    n = 4
    toplam_test = hata = 0
    for a, b, c0 in product(range(16), range(16), (0, 1)):
        s, c, _ = topla(ikili(a, n), ikili(b, n), c0)
        toplam_test += 1
        if c * 2 ** n + int(s, 2) != a + b + c0:
            hata += 1
    sonuc["kapsamli"] = (toplam_test, hata)


    retler = []
    for A, B, c0, aciklama in [("", "0101", 0, "boş girdi"),
                               ("01a1", "0011", 0, "geçersiz bit"),
                               ("101", "0011", 0, "farklı uzunluk"),
                               ("0101", "0011", 2, "geçersiz başlangıç eldesi")]:
        try:
            topla(A, B, c0)
            retler.append((aciklama, A, B, c0, "KABUL EDİLDİ (HATA)"))
        except ValueError as e:
            retler.append((aciklama, A, B, c0, f"ret: {e}"))
    sonuc["retler"] = retler

    # son eldeyi yok sayma: kayıp = c * 2^n ; en büyük kayıp örneği
    kayip = []
    for A, B, c0 in [("1111", "0001", 0), ("1111", "1111", 1), ("1000", "1000", 0)]:
        s, c, _ = topla(A, B, c0)
        gercek = int(A, 2) + int(B, 2) + c0
        kayip.append((A, B, c0, s, c, gercek, int(s, 2), gercek - int(s, 2)))
    sonuc["kayip"] = kayip
    return sonuc


# ------------------------------ Yaprak 2 ---------------------------------

def yaprak2():
    sonuc = {}
    islemler = ["1110-0011", "0011-1110", "1000-0001", "0101-0101", "0011-0000"]
    kayitlar = []
    for s in islemler:
        M, N = s.split("-")
        i1, b1, k1 = cikar_borc(M, N)
        i2, b2, k2 = cikar_tumleyen(M, N)
        kayitlar.append({"M": M, "N": N, "borc": (i1, b1, k1), "tum": (i2, b2, k2),
                         "ref": int(M, 2) - int(N, 2)})
    sonuc["islemler"] = kayitlar

    test = hata1 = hata2 = 0
    ilk_hata = None
    for m, k in product(range(16), range(16)):
        M, N = ikili(m, 4), ikili(k, 4)
        ref = m - k
        i1, b1, _ = cikar_borc(M, N)
        i2, b2, _ = cikar_tumleyen(M, N)
        test += 1
        ok1 = onlu_isaretli(i1, b1) == ref and (i1 == "0") == (ref == 0)
        ok2 = onlu_isaretli(i2, b2) == ref and (i2 == "0") == (ref == 0)
        hata1 += not ok1
        hata2 += not ok2
        if (not ok1 or not ok2) and ilk_hata is None:
            ilk_hata = (M, N)
    sonuc["kapsamli"] = (test, hata1, hata2, ilk_hata)

    # "Son elde yoksa sonuç mutlaka negatiftir" iddiası
    yanlis = []
    for m, k in product(range(16), range(16)):
        M, N = ikili(m, 4), ikili(k, 4)
        isaret, toplam, c = cikar_tumleyen_hatali(M, N)
        if (isaret == "-") != (m - k < 0):
            yanlis.append((M, N, toplam, c))
    sonuc["iddia_yanlis"] = yanlis
    sonuc["sifir_tumleyen"] = iki_tumleyeni("0000")
    # doğru yöntemde (M + N' + 1) N = 0 için son elde
    sonuc["dogru_n0"] = [(ikili(m, 4), cikar_tumleyen(ikili(m, 4), "0000")[2]["son_elde"])
                         for m in (0, 3, 15)]
    return sonuc


# ------------------------------ Yaprak 3 ---------------------------------

def yaprak3():
    sonuc = {}
```

```
    tablolar = []
    for A, B in [("1011", "101"), ("10111", "110"), ("11001", "1010")]:
        P, kismiler, cagri = carp(A, B)
        tablolar.append({"A": A, "B": B, "P": P, "kismiler": kismiler,
                         "katkilar": [int(pp, 2) for _, _, pp in kismiler],
                         "ref": int(A, 2) * int(B, 2)})
    sonuc["tablolar"] = tablolar

    testler = [("1011", "0", "çarpan sıfır"), ("1011", "1", "çarpan bir"),
               ("1011", "100", "çarpan 2^2"), ("101", "1000", "çarpan 2^3"),
               ("1011", "111", "çarpan tüm 1"), ("1111", "1111", "iki operand tüm 1"),
               ("0", "101", "çarpılan sıfır"), ("1", "1", "1 x 1"),
               ("11", "11", "n+m-1 karşı örneği"), ("110", "10001", "seyrek çarpan")]
    satirlar = []
    for A, B, ad in testler:
        P1, _, c1 = carp(A, B)
        P2, _, c2 = carp(A, B, sifirlari_atla=True)
        ref = int(A, 2) * int(B, 2)
        satirlar.append((ad, A, B, P1, P2, c1, c2, ref,
                         int(P1, 2) == ref and P1 == P2 and len(P1) == len(A) + len(B)
                         and c2 == B.count("1")))
    sonuc["testler"] = satirlar

    return sonuc


# ------------------------------ Yaprak 4 ---------------------------------

def yaprak4():
    sonuc = {}
    islemler = []
    for D, d in [("10110", "100"), ("1101", "10"), ("111", "11")]:
        r = bol(D, d, kesir_basamak=8)
        Dv, dv, Qv, Rv = int(D, 2), int(d, 2), int(r["Q"], 2), int(r["R"], 2)
        r.update(D=D, d=d, kontrol1=(Qv * dv + Rv == Dv), kontrol2=(0 <= Rv < dv),
                 ref=(Dv // dv, Dv % dv), deger=Dv / dv)
        islemler.append(r)
    sonuc["islemler"] = islemler

    sinir = []
    for D, d, ad, beklenen in [("0", "101", "sıfır bölünen", ("0", "0")),
                               ("000", "11", "sıfır bölünen (3 bit)", ("0", "0")),
                               ("11", "101", "bölen > bölünen", ("0", "11")),
                               ("101", "101", "eşit operandlar", ("1", "0")),
                               ("1011", "1011", "eşit operandlar", ("1", "0"))]:
        r = bol(D, d)
        sinir.append((ad, D, d, beklenen, (r["Q"], r["R"]), (r["Q"], r["R"]) == beklenen))
    try:
        bol("1010", "000")
        sinir.append(("sıfır bölen", "1010", "000", "ret", "KABUL EDİLDİ", False))
    except ZeroDivisionError as e:
        sinir.append(("sıfır bölen", "1010", "000", "ret", f"ret: {e}", True))
    sonuc["sinir"] = sinir

    hatali = []
    for D, d in [("10110", "100"), ("1101", "10"), ("100100", "11")]:
        Qh, Rh = bol_hatali(D, d)
        Q = bol(D, d)["Q"]
        hatali.append((D, d, Q, Qh, Rh,
                       int(Qh, 2) * int(d, 2) + int(Rh, 2) == int(D, 2)))
    sonuc["hatali"] = hatali

    return sonuc


def hepsi():
    return {1: yaprak1(), 2: yaprak2(), 3: yaprak3(), 4: yaprak4()}


if __name__ == "__main__":
    s = hepsi()
    print("Yaprak 1: 512 test, hata:", s[1]["kapsamli"][1])
    print("Yaprak 2: 256 çift, hata (borç, 2 tümleyeni):", s[2]["kapsamli"][1:3])
    print("Yaprak 3: testler uyumlu:", all(t[-1] for t in s[3]["testler"]))
    print("Yaprak 4: sınır testleri uyumlu:", all(t[-1] for t in s[4]["sinir"]))
```